

RESOLUCIÓN DE PROBLEMAS Y ALGORITMOS

Dra. Jessica Andrea Carballido

jac@cs.uns.edu.ar



Dpto. de Ciencias e Ingeniería de la Computación

UNIVERSIDAD NACIONAL DEL SUR

CLASE 6

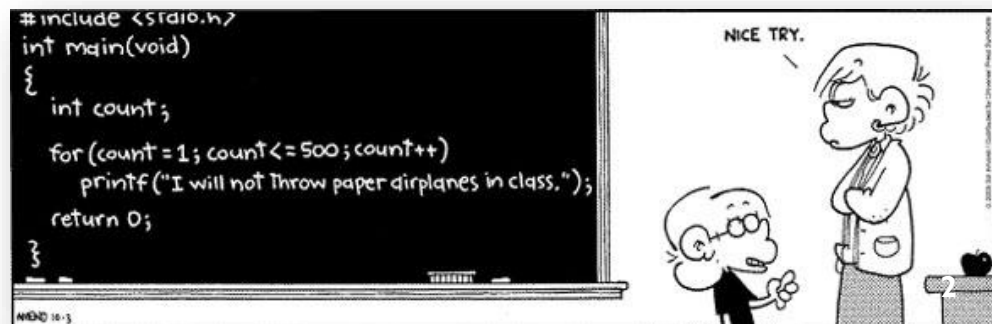


Iteración

La iteración permite modelar problemas en los cuales una secuencia de instrucciones se ejecuta repetidamente.

Pascal brinda mecanismos para expresar iteración dependiendo de:

- Una variable de control de tipo escalar que **recorre un intervalo automáticamente** y permite repetir N veces
- Una **expresión lógica**



Iteración con condición



Problema 1: Obtener el mayor de los dígitos de un número entero positivo **n**.

Por ejemplo, el mayor dígito de 6**8**75 es 8

Cuando resolvemos un problema, parte del desafío consiste en decidir qué **acciones primitivas** tenemos que usar.

Por el momento las primitivas que tenemos disponibles son las facilidades provistas por el lenguaje.

Para operar con enteros disponemos de los operadores
+ - * div y mod

Iteración con condición



Problema 1: Obtener el mayor de los dígitos de un número entero positivo n .

¿Podemos partir asumiendo que alguno de los dígitos de n es el mayor? ¿O que **0 es el mayor** y vamos comparando con todo el resto?

¿A cuál de los dígitos podemos acceder directamente?

✓ Asumimos inicialmente
que 0

es el mayor dígito

(ya que cualquier dígito que comparemos con 0,
será mayor).

✓ Tenemos que analizar
sí o sí TODOS LOS DÍGITOS del número.

Iteración con condición

Problema 1: Obtener el mayor de los dígitos de un número entero positivo n .



ASUMIMOS que 0 es EL MAYOR dígito.

Obtengo el último dígito de N

Si es mayor que EL MAYOR, actualizo EL MAYOR

Saco el último dígito de N

Obtengo el último dígito de N

Si es mayor que EL MAYOR, actualizo EL MAYOR

Saco el último dígito de N

Obtengo el último dígito de N

Si es mayor que EL MAYOR, actualizo EL MAYOR

Saco el último dígito de N

Hasta cuándo? ¡Hasta que se haya terminado el número N !

O sea, hasta que no haya más dígitos de N para analizar.

Iteración con condición



Algoritmo mayorDIGITO

DE: N (natural)

DS: eLMAYOR (un dígito entre 1 y 9)

Daux: ultDIG

Escribir el programa completo, seleccionar 4 casos de prueba y verificar

eLMAYOR $\leftarrow$ 0

Mientras ($N > 0$) hacer

 ultDIG $\leftarrow$ N mod 10

 si (ultDIG > eLMAYOR)

 entonces

 eLMAYOR $\leftarrow$ ultDIG

$N \leftarrow N \text{ div } 10$

Ejercicio: Se dice que un número es especial si es múltiplo del mayor de sus dígitos. Escriba un programa que reciba un número, valide que sea positivo y determine si es especial.

Iteración con condición



Problema I: Obtener el mayor de los dígitos de un número entero positivo n .

¿Podemos partir asumiendo que alguno de los dígitos de n es el mayor? ¿O que 0 es el mayor y vamos comparando con todo el resto?

¿A cuál de los dígitos podemos acceder directamente?

✓ Asumimos inicialmente
que 0

es el mayor dígito

(ya que cualquier dígito que comparemos con 0,
será mayor).

✓ ¿Tenemos que analizar
sí o sí **TODOS LOS DÍGITOS** del número?

Iteración con condición



Problema 2: Determinar si un dígito d entre 1 y 9 está en un número entero positivo N .

Por ejemplo:

- Si $d=3$ y $N= 605$, d no está en N .
- Si $d=3$ y $N= 305$, d está en N .
- Si $d=3$ y $N= 6035$, d está en N .
- Si $d=3$ y $N= 6053$, d está en N .
- Si $d=3$ y $N= 6303$, d está en N .

Iteración con condición



Problema 2: Determinar si un dígito d entre 1 y 9 está en un número entero positivo N .

Ver si el dígito buscado d
es igual al últ dígito de N
Eliminar últ dígito de N

¿Hasta cuándo?
¿En qué situaciones queremos
dejar de repetir estas acciones?

Iteración con condición



Problema 2: Determinar si un dígito d entre 1 y 9 está en un número entero positivo N .

Por ejemplo:

- Si $d=3$ y $N= 605$ d no está en N .
- Si $d=3$ y $N= 305$ d está en N .
- Si $d=3$ y $N= 6035$ d está en N .
- Si $d=3$ y $N= 6053$ d está en N .
- Si $d=3$ y $N= 6303$ d está en N .

Acciones
a repetir

Si $d=3$ y $N=261$

Comparar $d=3$ con 1 (el último dígito de N). Son distintos, elimino el último dígito de N .

Comparar $d=3$ con 6 (el último dígito de N). Son distintos, elimino el último dígito de N .

Comparar $d=3$ con 2 (el último dígito de N). Son distintos, elimino el último dígito de N .

N se terminó, no hay mas dígitos para comparar. Respuesta: el dígito d no está en N .

Iteración con condición



Problema 2: Determinar si un dígito d entre 1 y 9 está en un número entero positivo N .

Por ejemplo:

- Si $d=3$ y $N= 605$ d no está en N .
- Si $d=3$ y $N= 305$ d está en N .
- Si $d=3$ y $N= 6035$ d está en N .
- Si $d=3$ y $N= 6053$ d está en N .
- Si $d=3$ y $N= 6303$ d está en N .

Acciones
a repetir

Si $d=3$ y $N=6351$

Comparar $d=3$ con 1 (el últ. dígito de 6351). Son distintos, elimino el último dígito de N .

Comparar $d=3$ con 5 (el últ. dígito de 635). Son distintos, elimino el último dígito de N .

Comparar $d=3$ con 3 (el último dígito de 63). Son IGUALES, **encontré el dígito.**

Respuesta: el dígito d ESTA en N .

Iteración con condición



Problema 2: Determinar si un dígito d entre 1 y 9 está en un número entero positivo N .

Algoritmo digPresente

DE: N (natural), d (dígito entre 1 y 9)

DS: esta (v o f)

esta $\leftarrow$ falso

Mientras (**esta=falso**) hacer

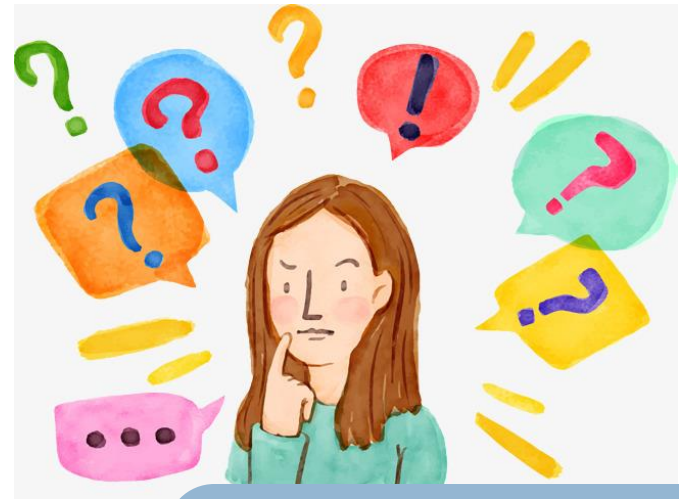
 Si $(N \bmod 10 = d)$

 entonces

esta $\leftarrow$ verdadero

 sino

$N \leftarrow N \div 10$



¿Si el dígito
NO está presente?
¡BUCLE INFINITO!

Iteración con condición



Problema 2: Determinar si un dígito d entre 1 y 9 está en un número entero positivo N .

Algoritmo digPresente

DE: N (natural), d (dígito entre 1 y 9)

DS: esta (v o f)

esta $\leftarrow$ falso

Mientras ($N > 0$) hacer

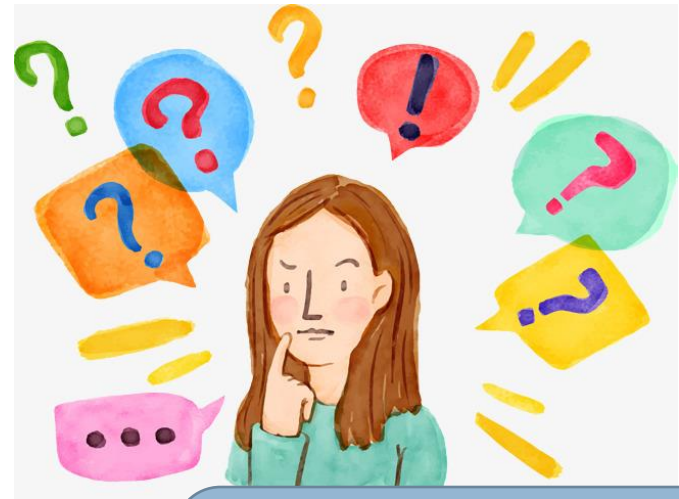
 si ($N \bmod 10 = d$)

 entonces

 esta $\leftarrow$ verdadero

 sino

$N \leftarrow N \div 10$



¿Si el dígito
está presente?
¡BUCLE INFINITO!

Iteración con condición



Problema 2: Determinar si un dígito d entre 1 y 9 está en un número entero positivo N .

Algoritmo digPresente

DE: N (natural), d (dígito entre 1 y 9)

DS: esta (v o f)

esta $\leftarrow$ falso

Mientras ($N > 0$) hacer

 si ($N \bmod 10 = d$)

 entonces

 esta $\leftarrow$ verdadero

$N \leftarrow N \div 10$



NO ES EFICIENTE

Iteración con condición



Problema 2: Determinar si un dígito d entre 1 y 9 está en un número entero positivo N .

Algoritmo digPresenteMIENTRAS

DE: N (natural), d (dígito entre 1 y 9)

DS: esta (v o f)

Doble condición de corte.

esta $\leftarrow$ falso

Mientras (($N > 0$) y ($\text{esta} = \text{falso}$)) hacer

 si ($N \bmod 10 = d$)

 entonces

 esta $\leftarrow$ verdadero

 sino

$N \leftarrow N \div 10$



Iteración con condición



```
program digPresenteWHILE;  
{Decide si un dígito d está en N }  
var N, d: integer;  
    esta: boolean;  
begin  
  {Entrada}  
  write('Ingrese N ');  
  readln(N); // VALIDAR QUE SEA NATURAL  
  write('Ingrese el dígito ');  
  readln(d); // VALIDAR QUE SEA UN DÍGITO (1 a 9)  
  ...
```


Iteración con condición



...

{Procesamiento}

esta:= false;

while ((N > 0) and (esta=false))

do

begin

if (N mod 10 = d)

then

esta := true

else

N := N div 10;

end;

...

(not esta)

Se modifica
el valor de "esta"

Se modifica
el valor de "N"

Iteración con condición



```
...  
{Procesamiento}  
  esta:= false;  
  while ((N > 0) and (esta=false))  
  do  
  begin  
    if (N mod 10 = d)  
    then  
      esta := true  
    else  
      N := N div 10;  
  end;  
  ...
```

HAY DOS CONDICIONES
DE CORTE.

*Mientras que “N tenga dígitos”
Y “aún no haya encontrado lo
que busco”*

Iteración con condición



```
program digPresenteWHILE;  
...  
begin  
  {Entrada} ...  
  {Procesamiento} ...  
  {Salida}  
  if esta (esta=true)  
  then  
    writeln(d, ' está en ', N)  
  else  
    writeln(d, ' no está en ', N) ;  
end.
```

¡¡Cuidado con los writeln!!
Se va a mostrar
un valor de N incorrecto ya que
iiiN fue modificado!!!
¿ COMO LO ARREGLARÍAS?



Iteración con condición

Algoritmo DigPresenteREPETIR

DE: N (natural), d (dígito entre 1 y 9)

DS: esta (lógico)

esta $\leftarrow$ falso

repetir

 esta $\leftarrow$ N mod 10 = d

 N $\leftarrow$ N div 10

Hasta (esta = verdadero) OR (N = 0)

Salimos si encontramos a d

OR Salimos si se terminó N

Cortar el bucle tan pronto como se pueda $\rightarrow$ **OPTIMIZAR**



Otra
forma

Determinar si la suma de los dígitos de un N es par
DE: N (natural)
DS: sumaPar
Daux: suma

```
suma ← 0
Mientras (N > 0) hacer
    [ suma ← suma + N mod 10
      N ← N div 10
```

```
Si (suma mod 2 = 0)
    Entonces
        sumaPar ← VERDADERO
```

```
Sino
    sumaPar ← FALSO
```

Determinar si la suma de los dígitos de un N natural es par.

Hacer traza para:

N = 723

Elegir la estructura de control repetitiva más adecuada.

OBJETIVO



Problema 3: A partir de dos enteros, a y b (con $a < b$), calcular la cantidad de números en el intervalo $[a, b]$ que son múltiplos de 2 y de 3 a la vez.

Ejemplos:

$a=12, b=20 \rightarrow$ la cantidad es 2 (el 12 y el 18).

$a=1, b=2 \rightarrow$ la cantidad es 0.

¿Datos de entrada?

¿Datos de salida?

¿Qué estructura de control deberíamos utilizar?



Problema 3: A partir de dos enteros, a y b (con $a < b$), calcular la cantidad de números en el intervalo $[a, b]$ que son múltiplos de 2 y de 3 a la vez.

Algoritmo cuantosEnIntervalo

DE: a, b (enteros)

DS: cant

Daux: num

$\text{cant} \leftarrow 0$

para num desde a hasta b hacer

 Si $(\text{num} \bmod 2 = 0)$ y $(\text{num} \bmod 3 = 0)$

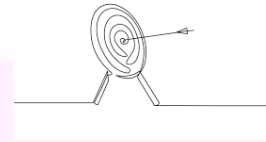
 ent $\text{cant} \leftarrow \text{cant} + 1$



Problema 3: A partir de dos enteros, a y b (con $a < b$), calcular la cantidad de números en el intervalo $[a, b]$ que son múltiplos de 2 y de 3 a la vez.

```
cant:=0;  
for num := a to b do  
    if (num mod 2=0) and (num mod 3=0)  
        then  
            cant:=cant + 1;
```


Iteración



```
Program CUANTOSenIntervalo;
var a, b, cant:, num integer;
begin
writeln('Ingrese los valores de a y b:');
readln(a, b); //VALIDAR a y b, a>0, b>a.
cant:=0;
for num := a to b do
    if (num mod 2=0) and (num mod 3=0)
        then
            cant:=cant + 1;
writeln('La cantidad de números que cumplen con la condición es: , cant);
end.
```

Elegir la estructura de control repetitiva más adecuada.

OBJETIVO



Problema 4: Dados dos enteros positivos, a y b (con $a < b$), se desea **determinar si hay algún número** en el intervalo $[a, b]$ que sea múltiplo de 2 y de 3.

Ejemplos:

$a=12, b=20.000 \rightarrow$ la respuesta es TRUE (el 12).

$a=100, b=3.000.000 \rightarrow$ la respuesta es TRUE (el 102).

$a=1, b=2 \rightarrow$ la respuesta es FALSE.

¿Datos de entrada?

¿Datos de salida?

¿Necesitamos sí o sí mirar todos los números en el intervalo?

¿Qué estructura de control deberíamos utilizar?



Problema 4: A partir de dos enteros positivos, a y b (con $a < b$) determinar si hay **algún** número en el intervalo $[a, b]$ que sea múltiplo de 2 y de 3.

Algoritmo hayAlguno

DE: a, b (enteros)

DS: hay (lógico)

hay $\leftarrow$ false

para num **desde** a **hasta** b **hacer**

 Si $(\text{num} \bmod 2 = 0)$ y $(\text{num} \bmod 3 = 0)$

 entonces

 hay $\leftarrow$ true;

El “PARA” recorre **SIEMPRE** el intervalo **completo** desde la expresión inicial a la expresión final.
No se puede cortar antes.



```
...  
hay:=false;  
num:=a;      doble condición  
while ((num<=b) and (hay=false)) do  
  begin  
    if (num mod 2=0) and (num mod 3=0)  
      then  
        hay:=true  
      else  
        num:=num+1;  
  end;  
...
```



HAY QUE OPTIMIZAR.

Si $a=6$ y $b=200000$, no necesito recorrer todo el intervalo. Al chequear el 6 ya puedo decir que hay al menos un número que cumple con la condición.

!!!ES EFICIENTE!!!

Legibilidad

- Nombres claros y uso consistente de mayúsculas y minúsculas.
- Indentación y espaciado.
- Usar comentarios con buen criterio: los comentarios excesivos pueden resultar contraproducentes; el comentario debe describir características del problema o de la solución, no del lenguaje de programación.
- No modificar valores de NINGUNA DE las variables que controlan los bucles *for*. Ni de la variable de control, ni de las que puedan participar de la expresión inicial ni de la expresión final.
- **No podemos asumir ningún valor** para la variable de control una vez que estamos fuera del bucle *for*.

Errores

Un programa puede contener errores:

- **estáticos o de compilación**, los detecta el compilador, el programa no llega a traducirse a código máquina y por lo tanto, a ejecutarse;
- **dinámicos o de ejecución**, el programa se comienza a ejecutar pero termina anormalmente, o queda en un bucle infinito;
- **lógicos o de aplicación**, el programa se ejecuta por completo, concluye “con éxito” (llega exitosamente a ejecutar el *end.*) pero no realiza lo esperado. Da un resultado incorrecto. Deberían ser detectados durante la verificación.



Errores

ERRORES DE COMPILACION

Antes de que el programa se ejecute se detectan errores como:

- Un identificador se usa sin haberse declarado (semántico).
- Incompatibilidad de tipos en una expresión (semántico).
- Los paréntesis en una expresión no están bien balanceados (sintáctico).
- Modificar el valor de la variable de control del FOR (semántico).

Error: Illegal assignment to for-loop variable "i"

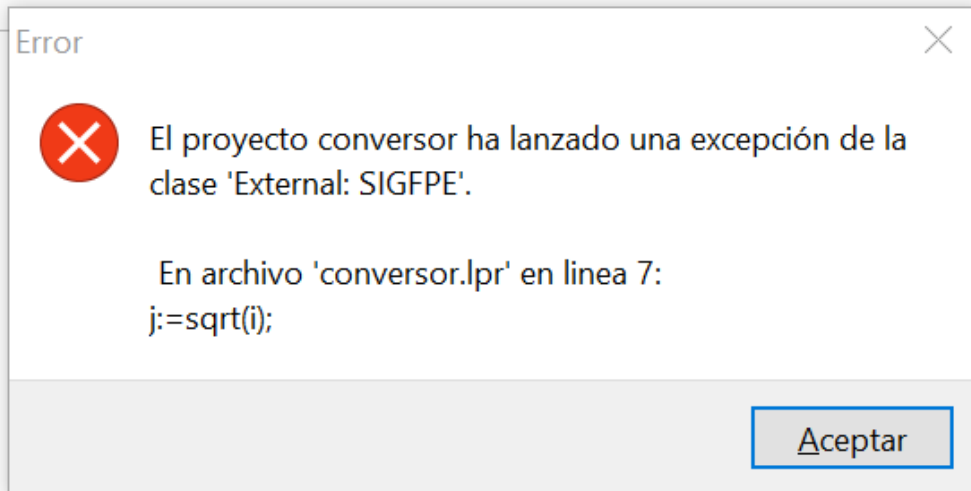


Errores

ERRORES EN EJECUCION

Durante la ejecución de un programa se detectan errores como:

- El operador de división con divisor **0**.
- El operador `sqrt` aplicado a un argumento **negativo**.
- La **lectura de un dato de un tipo** diferente al tipo esperado (se espera un entero y se ingresa una letra).



Errores

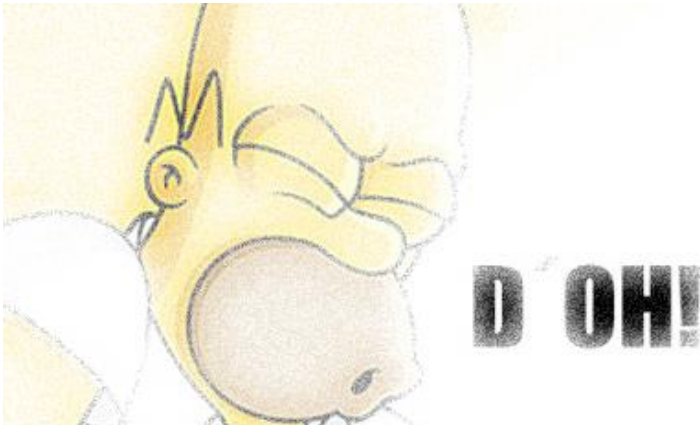
ERRORES LOGICOS

Los **errores lógicos (o de aplicación)** son errores que impiden que el programa haga lo que estaba previsto.

El código puede compilarse y ejecutarse sin errores, pero el resultado de una operación puede generar un resultado general no esperado.

Estos errores son los más difíciles de detectar y corregir.

Un bucle infinito también es considerado error lógico.



```

1 program sumaDigitos;
. var N, d, suma: integer;
. // Este programa calcula la suma
. // de los digitos de un entero
5 // ingresado por el usuario
. begin
. writeln('Ingrese un entero: ');
. readln(N);
. suma:=0;
10 while (N>0) do
. begin
.     d:=N mod 10;
.     suma:=suma*d;
.     N:=N div 10;
15 end;
. writeln('La suma es: ', suma);
. readln;readln;
. end.

```

azarus v1.4.0 - conversor (depurando ..

editar Buscar Ver Fuente Proyecto

Standard Additional Con

Ok Abc Tab

C:\Users\Jessi\Dropbox\UNI\F

Ingrese un entero:

26

La suma es: 0

error LOGICO

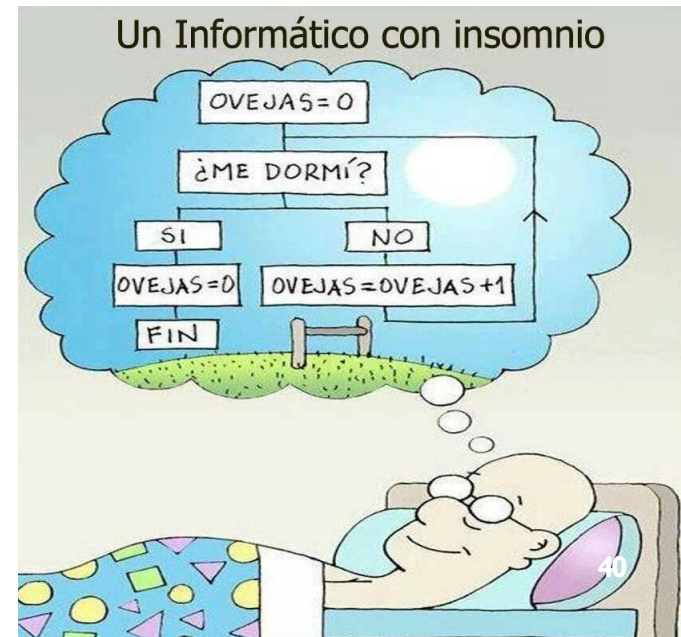
Estructuras de control

Las estructuras de control permiten establecer el flujo de ejecución, esto es, el **orden** en el cual se ejecutarán las instrucciones.

- **SECUENCIA**
- **SELECCIÓN**
- **ITERACIÓN**

Con estas tres estructuras se pueden escribir **todos los programas y aplicaciones posibles**.

Si bien algunos lenguajes de programación tienen un mayor repertorio de estructuras de control, estas pueden ser construidas mediante las tres básicas citadas.



PROGRAMACIÓN ESTRUCTURADA

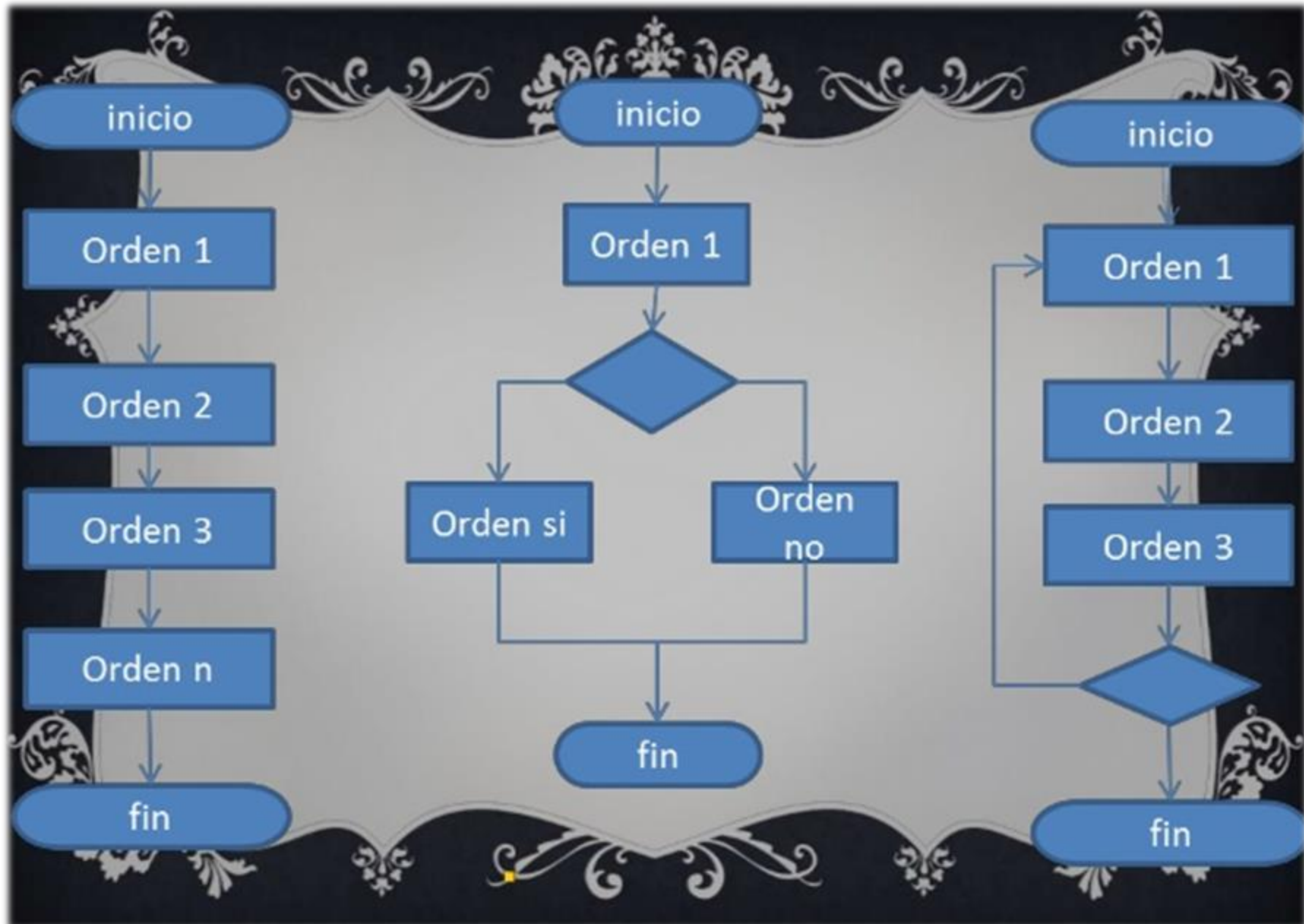
- ❖ La programación estructurada es un paradigma de programación orientado a mejorar la claridad, calidad y tiempo de desarrollo de un programa de computadora recurriendo únicamente a subrutinas y tres estructuras básicas: secuencia, selección (if y case) e iteración (bucles for, repeat y while).
- ❖ Se considera **innecesario y contraproducente** el uso de la instrucción de transferencia (GOTO), que podría conducir a código “espagueti”, mucho más difícil de seguir y de mantener, y fuente de numerosos errores de programación.

PROGRAMACIÓN ESTRUCTURADA

- ❖ Código “*espagueti*”: término peyorativo para los programas de computación que tienen una estructura de control de flujo compleja e incomprensible.



```
1 C    A weird program for calculating Pi written in
2 C    From: Fink, D.G., Computers and the Human Min
3
4      PROGRAM PI
5      DIMENSION TERM(100)
6      N=1
7      3 TERM(N)=((-1)**(N+1))*(4./(2.*N-1.))
8      N=N+1
9      IF (N-101) 3,6,6
10     N=1
11     7 SUM98 = SUM98+TERM(N)
12     WRITE(*,28) N, TERM(N)
13     N=N+1
14     IF (N-99) 7, 11, 11
15     11 SUM99=SUM98+TERM(N)
16     SUM100=SUM99+TERM(N+1)
17     IF (SUM98-3.141592) 14,23,23
18     14 IF (SUM99-3.141592) 23,23,15
19     15 IF (SUM100-3.141592) 16,23,23
20     16 AV89=(SUM98+SUM99)/2.
21     AV90=(SUM99+SUM100)/2.
22     COMANS=(AV89+AV90)/2.
23     IF (COMANS-3.1415920) 21,19,19
24     19 IF (COMANS-3.1415930) 20,21,21
25     20 WRITE(*,26)
26     GO TO 22
27     21 WRITE(*,27) COMANS
28     STOP
29     23 WRITE(*,25)
30     GO TO 22
31     25 FORMAT('ERROR IN MAGNITUDE OF SUM')
32     26 FORMAT('PROBLEM SOLVED')
33     27 FORMAT('PROBLEM UNSOLVED', F14.6)
34     28 FORMAT(I3, F14.6)
35     END
36
```

EJEMPLO CÓDIGO SPAGUETTI

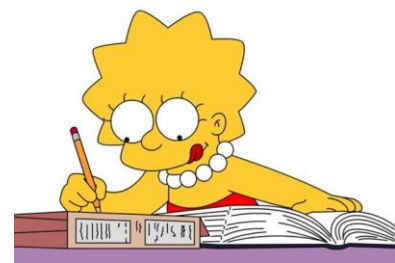
```
1. leer(a)
2. leer(b)
3. si (a<b) goto(1)
4. c←a+b
5. si (c<0) goto(10)
6. goto(8)
7. mostrar('nunca')
8. goto(1)
9. si (b<123) goto(2)
10. fin
```





**Study
at home**

REPASO Y PRACTICA



Iteración con condición



¿Qué es una **sucesión** numérica?

Es una lista de números en donde la relación entre un número y el siguiente es siempre la misma.

Términos: Cada uno de los números de la sucesión

Patrón (o regla de formación): Relación entre cada par de elementos de la sucesión.

Una **serie** es un conjunto de números (llamados términos) ordenados según un **patrón** o regla de formación, **unidos por una operación, comúnmente una suma.**

Iteración con condición



- Se llama sucesión a un conjunto de números dispuestos uno a continuación de otro.

$$a_1, a_2, a_3, \dots, a_n$$

$$3, 6, 9, \dots, 3n$$

Los números $a_1, a_2, a_3, \dots$ se llaman términos de la sucesión.

El subíndice indica el lugar que el término ocupa en la sucesión.

El término general es a_n es un criterio que nos permite determinar cualquier término de la sucesión.

Ejemplos sucesiones (armar los primeros 5 elementos con la información del término general)

Término general

- $2n+1$
- $(n-1)*2$
- $7n-10$
- $3*(n+2)$

Iteración con condición



Ejemplos sucesiones (armar los primeros 5 elementos con la información del término general)

Término general

- a. $2n+1$
- b. $(n-1)*2$
- c. $7n-5$
- d. $3*(n+2)$

Sucesión para a.

3 (n=1), 5 (n=2), 7 (n=3), 9 (n=4), 11 (n=5)

Serie uniendo con sumas, calculada hasta $n = 5$:

$$3+5+7+9+11 = 35$$

Problema: A partir de un valor n calcular la serie

$$1^2 + 2^2 + 3^2 + \dots + n^2$$

¿Cuáles son los datos de entrada?

¿Cuáles son los datos de salida?

¿Cuáles pueden ser los casos de prueba?

Nuevo concepto:
ACUMULADOR

Si $n=5$ debo calcular $1^2 + 2^2 + 3^2 + 4^2 + 5^2$

Acumulación de **sumas** sucesivas:

n veces

$0 + 1^2$
.. $+ 2^2$
.. $+ 3^2$
.. $+ n^2$

El valor de la base en cada término
varía desde 1 hasta n .

varía desde 1 hasta n .

Algoritmo SumaDeSerie

DE: n

DS: serie

Comienzo

serie $\leftarrow$ 0

para base desde 1 hasta n hacer

 serie $\leftarrow$ serie + base * base

Fin



serie := 0;

for base := 1 to n do

 serie := serie + sqr(base);

La **variable de control** toma valores en el rango que determinan la **expresión inicial** y la **expresión final**.

```
program serie;  
{Computa  $1^2 + 2^2 + 3^2 + \dots + n^2$  }  
var base, n, serie: integer;  
begin  
  write ('Ingrese el valor de la cantidad de terminos n ');  
  readln (n);  
  serie := 0;  
  for base:= 1 to n  
  do  
    serie := serie + sqr(base);  
  writeln ('El valor es ', serie);  
end.
```

¿Qué pasa si el usuario ingresa un valor menor o igual a 0?

Ejercicio 1: A partir de un valor n (positivo) calcular la serie

$$n*1^2 + n*2^2 + n*3^2 + \dots + n*n^2$$

Ejercicio 2: A partir de dos valores a y b (positivos con $a < b$) calcular la serie

$$\sum_{i=a}^b 3$$



Ejercicio 3: A partir de dos valores a y b (positivos con $a < b$) calcular la serie

$$\sum_{i=a}^b i * 5$$

Ejercicio 4: A partir de un valor n (mayor a 1) calcular la serie

$$\sum_{i=1}^n \text{sqrt}(i)$$

Iteración con contador



for *variable de control* := *valor inicial* to *valor final* do
instrucción simple o compuesta

- La *variable de control* debe ser de un **tipo ordinal**. Usaremos enteros o caracteres.
- El *valor inicial* y el *valor final* puede ser una constante, una variable o una expresión del mismo tipo que la variable de control.
- La instrucción simple o compuesta es el **bloque iterativo** y puede no ejecutarse.
- El bloque iterativo no puede modificar el valor de la variable de control.
- Tampoco deberían modificarse el resto de las variables que afectan al valor inicial y al valor final.

Iteración con contador



for *variable de control* := *valor inicial* to *valor final* do
instrucción simple o compuesta

```
program bucleFor;  
var i: integer;  
begin  
  for i:= 1 to 5  
  do  
    writeln(i);  
end.
```

```
program bucleFor;  
var i: integer;  
begin  
  for i:= 0 to 5  
  do  
    writeln(i);  
end.
```

Iteración con contador



for *variable de control* := *valor inicial* to *valor final* do
instrucción simple o compuesta

```
program bucleFor;  
var i: integer;  
begin  
  for i:= 10 to 5  
  do  
    writeln (i);  
end.
```

```
program bucleFor;  
var i: integer;  
begin  
  for i:= 10 downto 5  
  do  
    writeln (i);  
end.
```

Iteración con contador



for *variable de control* := *valor inicial* to *valor final* do
instrucción simple o compuesta

```
program bucleFor;  
var i: integer;  
begin  
  for i:= -1 to 1  
  do  
    writeln (i);  
end.
```

```
program bucleFor;  
var i: char;  
begin  
  for i:= 'a' to 'm'  
  do  
    writeln ('Letra:', i);  
end.
```

Iteración con contador



for *variable de control* := *valor inicial* to *valor final* do
instrucción simple o compuesta

```
program bucleFor;  
var i, n: integer;  
begin  
  n:=5;  
  for i:= 1 to n  
  do  
    begin  
      writeln (i, n);  
      if i = 3 then i := 10;  
      writeln (i, n);  
    end;  
  end;  
end.
```

```
project1.lpr  
1  program buclefor;  
  .  var i, n: integer;  
  .  begin  
  .    n:=5;  
5    for i:= 1 to n  
  .    do  
  .      begin  
  .        writeln (i, n);  
9      if i = 3 then i := 10;  
10     writeln (i, n);  
  .      end;  
  .      readln();  
  .    end.  
15
```

Ventana de Mensajes

- project1.lpr(9,23) Error: Illegal assignment to for-loop variable "i"
- project1.lpr(16) Fatal: There were 1 errors compiling module, stopping

Error al compilar

Iteración con contador



for *variable de control* := *valor inicial* to *valor final* do
instrucción simple o compuesta

```
program bucleFor;  
var i, n: integer;  
begin  
    n:=5;  
    for i:= 1 to n  
    do  
        begin  
            writeln (i, n);  
            if i = 3 then n := 4;  
            writeln (i, n);  
        end;  
    end;  
end.
```

```
*project1.lpr  
1  program buclefor;  
  .  var i, n: integer;  
  .  begin  
  .      n:=5;  
  .      for i:= 1 to n  
  .      do  
  .          begin  
  .              writeln (i, n);  
  .              if i = 3 then n := 4;  
  .              writeln (i, n);  
  .          end;  
  .      readln();  
  .  end.  
15
```

C:\Users\VC
15
15
25
25
35
34
44
44
54
54

Oscurece la legibilidad del programa

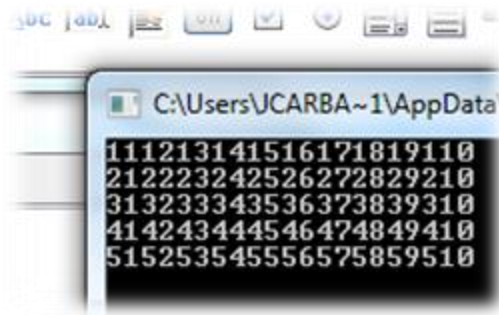
NO LO VAMOS A PERMITIR EN ESTA MATERIA

Iteración con contador

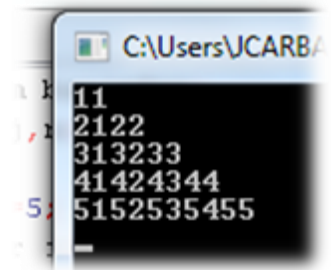


for *variable de control* := *valor inicial* to *valor final* do
instrucción simple o compuesta

```
program bucleFor;  
var i, j, n: integer;  
begin  
  n:=5;  
  for i:= 1 to n  
  do  
    begin  
      for j:= 1 to 10  
      do  
        write (i, j);  
        writeln;  
      end;  
    end;  
  end.
```



```
program bucleFor;  
var i,j,n: integer;  
begin  
  n:=5;  
  for i:= 1 to n  
  do  
    begin  
      for j:= 1 to i  
      do  
        write (i, j);  
        writeln;  
      end;  
    end;  
  end.
```



Iteración con contador



Escribir un programa que muestre el código ASCII de todas las letras mayúsculas.

¿Cuáles son los datos de entrada?

¿Cuáles son los datos de salida?

¿Cuáles pueden ser los casos de prueba?

A= 65

B= 66

C= 67

..

X= 88

Y= 89

Z= 90

				32		33	!	34	"	35	#	36	\$	37	%	38	&	39	'
40	(	41	)	42	*	43	+	44	,	45	-	46	.	47		48	0	49	1
50	2	51	3	52	4	53	5	54	6	55	7	56	8	57	9	58	:	59	.
60	<	61	=	62	>	63	?	64	@	65	A	66	B	67	C	68	D	69	E
70	F	71	G	72	H	73	I	74	J	75	K	76	L	77	M	78	N	79	O
80	P	81	Q	82	R	83	S	84	T	85	U	86	V	87	W	88	X	89	Y
90	Z	91	[	92	\	93	]	94	^	95	_	96		97	a	98	b	99	c
100	d	101	e	102	f	103	g	104	h	105	i	106	j	107	k	108	l	109	m
110	n	111	o	112	p	113	q	114	r	115	s	116	t	117	u	118	v	119	w
120	x	121	y	122	z	123	{	124		125	}	126	~	127		128	Ç	129	ü

Iteración con contador



Escribir un programa que muestre el código ASCII de todas las letras mayúsculas.

```
writeln ('A', '=', ord('A'));  
writeln ('B', '=', ord('B'));  
writeln ('C', '=', ord('C'));  
...
```

La **letra** “varía” tomando valores '**A**', '**B**', ... hasta '**Z**'.

Podemos usar una única instrucción reemplazando la **constante literal** con una **variable**.

```
writeln (letra, '=', ord(letra));
```

Iteración con contador



Escribir un programa que muestre el código ASCII de las letras mayúsculas.

```
program ordinalesLetras;  
var letra: char;  
begin  
  for letra := 'A' to 'Z' do  
    writeln (letra, ' = ', ord(letra));  
  end.
```

La **variable de control** toma valores en el rango que determinan los caracteres 'A' y 'Z'.

Iteración con condición



while *expresión lógica* do
sentencia simple o compuesta

```
program bucleWhile;  
var i, n: integer;
```

```
begin  
  n:=5; i:=1;  
  while (i < n)  
  do  
    begin  
      writeln (i, n);  
      i:= i+1;  
    end;  
end.
```

```
program bucleWhile;  
var i, n: integer;
```

```
begin  
  n:=5; i:=n;  
  while (i < n)  
  do  
    begin  
      writeln (i, n);  
      i := i+1;  
    end;  
end.
```

Iteración con condición



while *expresión lógica* do
sentencia simple o compuesta

```
program bucleWhile;  
var i, n: integer;
```

```
begin  
  n:=5;  i:=1;  
  while (i < n)  
  do  
    writeln (i, n);  
end.
```

```
program bucleWhile;  
var i, n: integer;
```

```
begin  
  n:=5;  i:=n;  
  while (i >= n)  
  do  
    begin  
      writeln (i, n);  
      i := i+1;  
    end;  
end.
```